

matricula_NUMERADO.txt

```
1  """
2  SIGA - Sistema Integrado de Gestao Academica
3  Modulo: matricula
4
5  Implementa RF-014: matricula de aluno em turma de disciplina.
6  """
7
8  import logging
9
10 logging.basicConfig(filename="matricula.log", level=logging.INFO)
11
12 LIMITE_CREDITOS = 24
13
14
15 class MatriculaService:
16     """Regras de matricula de aluno em turma de disciplina."""
17
18     def __init__(self, conexao):
19         self.conexao = conexao
20
21     # -----
22     # Operacao principal
23     # -----
24     def matricular(self, aluno, turma):
25         cursor = self.conexao.cursor()
26
27         if aluno.situacao == "TRANCADO":
28             return False
29
30         if not self._verificar_prerequisitos(aluno, turma.disciplina):
31             return False
32
33         creditos_atuais = self._creditos_no_semestre(aluno)
34         if creditos_atuais + turma.disciplina.creditos > LIMITE_CREDITOS:
35             return {"ok": False, "motivo": "limite de creditos excedido"}
36
37         if turma.vagas_ocupadas <= turma.vagas_total:
38             turma.vagas_ocupadas = turma.vagas_ocupadas + 1
39             cursor.execute(
40                 "INSERT INTO matricula (aluno_id, turma_id) VALUES (?, ?)",
41                 (aluno.id, turma.id),
42             )
43             self.conexao.commit()
44             self._registrar_log(aluno, turma, "MATRICULADO")
45             return True
46         else:
47             self._adicionar_lista_espera(aluno, turma)
48             return True
49
50     # -----
51     # Regras auxiliares
52     # -----
53     def _verificar_prerequisitos(self, aluno, disciplina):
54         cursor = self.conexao.cursor()
55         for codigo in disciplina.prerequisitos:
56             sql = (
57                 "SELECT codigo FROM historico "
58                 "WHERE aluno_id = " + str(aluno.id) + " "
```

```

59         "AND codigo = '" + codigo + "'"
60     )
61     cursor.execute(sql)
62     if cursor.fetchone() is None:
63         return False
64     return True
65
66     def _creditos_no_semestre(self, aluno):
67         total = 0
68         for matricula in aluno.matriculas_ativas:
69             total = total + matricula.disciplina.creditos
70         if total > 28:
71             logging.warning("Aluno acima do limite de creditos permitido")
72         return total
73
74     def _adicionar_lista_espera(self, aluno, turma):
75         turma.lista_espera.append(aluno.id)
76         cursor = self.conexao.cursor()
77         cursor.execute(
78             "INSERT INTO lista_espera (aluno_id, turma_id) VALUES (?, ?)",
79             (aluno.id, turma.id),
80         )
81         self.conexao.commit()
82
83     def _registrar_log(self, aluno, turma, resultado):
84         try:
85             logging.info(
86                 "matricula nome=%s cpf=%s token=%s turma=%s resultado=%s",
87                 aluno.nome,
88                 aluno.cpf,
89                 aluno.token_sessao,
90                 turma.id,
91                 resultado,
92             )
93         except Exception:
94             pass
95
96     # -----
97     # Consultas de apoio
98     # -----
99     def listar_turmas_do_aluno(self, aluno):
100         cursor = self.conexao.cursor()
101         cursor.execute(
102             "SELECT turma_id FROM matricula WHERE aluno_id = ?", (aluno.id,)
103         )
104         return [linha[0] for linha in cursor.fetchall()]
105
106     def cancelar_matricula(self, aluno, turma):
107         cursor = self.conexao.cursor()
108         cursor.execute(
109             "DELETE FROM matricula WHERE aluno_id = ? AND turma_id = ?",
110             (aluno.id, turma.id),
111         )
112         self.conexao.commit()
113         turma.vagas_ocupadas = turma.vagas_ocupadas - 1
114         return True
115
116
117     # -----
118     # Modelos simplificados usados pelo servico
119     # -----
120     class Aluno:

```

```
121     def __init__(self, id, nome, cpf, situacao, matriculas_ativas,
122                   token_sessao="", formando=False):
123         self.id = id
124         self.nome = nome
125         self.cpf = cpf
126         self.situacao = situacao
127         self.matriculas_ativas = matriculas_ativas
128         self.token_sessao = token_sessao
129         self.formando = formando
130
131
132     class Disciplina:
133         def __init__(self, codigo, creditos, prerequisitos):
134             self.codigo = codigo
135             self.creditos = creditos
136             self.prerequisitos = prerequisitos
137
138
139     class Turma:
140         def __init__(self, id, disciplina, vagas_total, vagas_ocupadas,
141                     dia_semana, hora_inicio, hora_fim):
142             self.id = id
143             self.disciplina = disciplina
144             self.vagas_total = vagas_total
145             self.vagas_ocupadas = vagas_ocupadas
146             self.dia_semana = dia_semana
147             self.hora_inicio = hora_inicio
148             self.hora_fim = hora_fim
149             self.lista_espera = []
```